

File Uploading in Node.js and Express Using Multer

This tutorial explains how to upload images and other file types using Node.js, Express.js, and the Multer package. It covers installation, configuration, storage options, limits, file filters, error handling, and handling single, multiple, and multi-field uploads. All view examples use **Twig**.

1. Introduction to File Uploading in Express.js

Uploading files (images, PDFs, Excel, Word documents, videos) in Express.js requires a middleware that can handle multipart form data. Multer is widely used because:

- It supports defining storage destination and file names.
 - Allows validation rules such as file size limits, number of files, and allowed file formats.
 - Allows multiple uploads and multi-field uploads.
-

2. Key Multer Configuration Options

Multer accepts three major configuration options:

2.1 Storage (Required)

Defines:

- **destination:** folder where files should be stored
- **filename:** logic for generating stored file names

2.2 Limits (Optional)

Allows setting:

- **fileSize:** maximum allowed size
- **files:** maximum number of files
- **fields:** max fields if using multiple upload fields

2.3 File Filter (Optional)

Useful for validating:

- file type (image only, PDF only, etc.)
 - MIME-type specific restrictions (e.g., only JPG/PNG)
-

3. Installing and Importing Multer

Install

```
npm install multer
```

Import in your Express app

```
import multer from "multer";  
import path from "path";
```

4. Defining Storage Configuration

```
const storage = multer.diskStorage({  
  destination: (req, file, cb) => {  
    cb(null, "./uploads"); // path to uploads folder  
  },  
  filename: (req, file, cb) => {  
    const uniqueName = Date.now() + path.extname(file.originalname);  
    cb(null, uniqueName);  
  }  
});
```

5. Adding File Size Limits

Example: Max upload size = **3 MB**

```
const upload = multer({  
  storage: storage,  
  limits: {  
    fileSize: 1024 * 1024 * 3  
  }  
});
```

6. Adding File Filters (Validate File Type)

6.1 Allow Only Images (Any Format)

```
const fileFilter = (req, file, cb) => {  
  if (file.mimetype.startsWith("image/")) {
```

```
    cb(null, true);
  } else {
    cb(new Error("Only images are allowed"), false);
  }
};
```

6.2 Allow Only JPG and PNG

```
const fileFilter = (req, file, cb) => {
  if (
    file.mimetype === "image/jpeg" ||
    file.mimetype === "image/png"
  ) {
    cb(null, true);
  } else {
    cb(new Error("Only JPG or PNG allowed"), false);
  }
};
```

Attach to Multer

```
const upload = multer({
  storage,
  limits,
  fileFilter
});
```

7. Creating Required Upload Folder

```
/uploads
```

(This must exist before uploading.)

8. Front-End Form (Twig)

Multer requires the form to submit **multipart/form-data**.

Example `upload.twig`

```
<form action="/submit-form" method="POST" enctype="multipart/form-data">
  <div>
    <label>User Name</label>
    <input type="text" name="username" />
  </div>

  <div>
    <label>Select File</label>
    <input type="file" name="userFile" />
  </div>

  <button type="submit">Upload</button>
</form>
```

9. Single File Upload Route

```
app.post("/submit-form", upload.single("userFile"), (req, res) => {
  res.send(req.file); // shows file info
});
```

10. Multiple File Upload (Same Field)

Front-end (Twig):

```
<input type="file" name="userFiles" multiple />
```

Backend:

```
app.post("/upload-multiple", upload.array("userFiles", 5), (req, res) => {
  res.send(req.files);
});
```

11. Multi-Field Upload (Different Fields)

Twig:

```
<form action="/multi-upload" method="POST" enctype="multipart/form-data">
  <label>Profile Image</label>
  <input type="file" name="profilePic" />

  <label>Documents</label>
  <input type="file" name="docs" multiple />

  <button type="submit">Upload</button>
</form>
```

Backend:

```
app.post("/multi-upload",
  upload.fields([
    { name: "profilePic", maxCount: 1 },
    { name: "docs", maxCount: 3 }
  ]),
  (req, res) => {
    res.send(req.files);
  }
);
```

12. Multi-Field Upload with Different Restrictions

Example:

- `profilePic` → only JPG and PNG
- `docs` → only PDFs

```
const fileFilter = (req, file, cb) => {
  if (file.fieldname === "profilePic") {
    if (file.mimetype === "image/jpeg" || file.mimetype === "image/png") {
      cb(null, true);
    } else {
      cb(new Error("Only JPG/PNG allowed for profile picture"), false);
    }
  }

  else if (file.fieldname === "docs") {
    if (file.mimetype === "application/pdf") {
      cb(null, true);
    } else {
      cb(new Error("Only PDF allowed for documents"), false);
    }
  }
}
```

```
    else {
      cb(new Error("Unknown upload field"), false);
    }
  };
};
```

Attach:

```
const upload = multer({
  storage,
  limits,
  fileFilter
});
```

13. Error Handling Middleware

Custom Error Handler

```
app.use((err, req, res, next) => {

  // Case 1: File missing
  if (!req.file && !req.files) {
    return res.status(400).send("No file uploaded");
  }

  // Case 2: Multer-specific errors
  if (err instanceof multer.MulterError) {

    // Too many files
    if (err.code === "LIMIT_UNEXPECTED_FILE") {
      return res.status(400).send("Too many files uploaded");
    }

    // File too large
    if (err.code === "LIMIT_FILE_SIZE") {
      return res.status(400).send("File too large");
    }

    return res.status(400).send("Multer error: " + err.message);
  }

  // General server errors
  return res.status(500).send("Something went wrong: " + err.message);
});
```

14. File Information Returned by Multer

Multer returns an object containing:

```
{
  filename: 'userFile',
  originalname: 'photo.jpg',
  encoding: '7bit',
  mimetype: 'image/jpeg',
  destination: './uploads',
  filename: '1704648819275.jpg',
  path: 'uploads/1704648819275.jpg',
  size: 21034
}
```

To store in MongoDB/MySQL, only the **filename** field is typically saved.

15. Handling Large Files

When uploading a file larger than the configured limit, Multer throws:

```
MulterError: File too large
```

Handled using error middleware shown above.

16. Summary of Multer Features Covered

Implemented:

- Single file upload
- Multiple file upload
- Multi-field upload
- File size limit
- MIME-type filtering
- Custom filename generation
- Error handling for:
 - No file sent

- Wrong file type
- Too many files
- File too large

Mandatory and Optional:

- **storage**: Mandatory
- **limits**: Optional
- **fileFilter**: Optional